

ECO PULSE

SMART AC CONTROLLER

Firmware and System Architecture Report

Stage: Production / Field-Deployed Firmware (1.0.15)

Platform: ESP32 (Espressif) — Arduino / FreeRTOS

Classification: Confidential

Executive Summary

This document presents the complete firmware and system architecture for the **ECO PULSE** smart air-conditioner controller. It serves as the primary technical reference for the engineering team during development and maintenance, and for the client team for architectural review and alignment.

The report covers every layer of the firmware stack, from hardware component selection and requirements scope through to the firmware layer model, RTOS task and thread architecture, data flow, power architecture, communication architecture, the operating-mode state machine, error handling, and security. All decisions documented here represent the agreed architecture baseline for the current production firmware. Tuning of parameters such as eco/off timers, radar detection range, the "no-one" presence-hold window, schedule segments, and reconnect back-off intervals is permitted during bring-up and field tuning, but no architectural or feature-level changes are expected beyond what is defined in this document without a formal revision.

This document does **not** cover the cloud/back-end MQTT broker implementation, the customer mobile or web front-end that issues commands, the AC unit's own internal electronics, or PCB mechanical design. Those are intentionally out of scope for this report.

Any section marked with a **Note** or **Assumption** indicates a decision that was made based on current project information and may be revisited in a future hardware or firmware revision.

1. Requirements Review and Scope Freeze

This section outlines the technical requirements, functional scope, and architectural Specifications for **ECO PULSE**, an ESP32-based smart controller that automates a standard infrared-controlled split air conditioner based on **room occupancy** (mmWave presence radar) and a **7-day time schedule**, with remote control and telemetry over the cloud.

The device is designed as a mains-powered, always-on retrofit controller. It learns the AC unit's IR remote, and then drives the unit through learned/universal IR commands while sensing room presence, temperature and humidity. The primary objective of this baseline is reliable, unattended automation that degrades gracefully when the network or clock is unavailable.

Critical Design Note

The radar hardware is the **14-gate HLK-LD2412** 24 GHz mmWave module. The firmware drives it through a **local fork** of the MyLD2410 driver (originally a 9-gate LD2410 library) adapted in lib/LD2412/ to expose all 14 gates for per-gate energy reporting and range control. The upstream 9-gate library must **not** be re-introduced.

Hardware Architecture Requirements

ECO PULSE uses a single-board ESP32 design with externally wired sensors and an IR front-end. All electronic components conform to the following scope.

Sensor & Component Bill of Materials (BOM)

The following components define the "locked" hardware scope for the production build:

Component	Model / Spec	Function	Requirement Note
MCU	ESP32 DoIT DevKit (ESP32-WROOM-32)	Main controller	Dual-core Xtensa LX6 @ 240 MHz, Wi-Fi + BT, FreeRTOS.
Presence Radar	HLK-LD2412 (24 GHz mmWave)	Occupancy / motion	14 detection gates; UART @ 115200; per-gate moving/static energy.
Climate Sensor	HDC1080 (HDC1000-class)	Temperature + humidity	I ² C @ 0x40; periodic ~10 s sampling.
IR Transmitter	IR LED via MOSFET driver	AC command output	38 kHz; learned custom buttons + universal AC protocol fallback.
IR Receiver	IR demodulator (always-on)	Remote-press detection / learning	Detects foreign remote presses → manual-override reporting.

IR Power Switch	N-channel MOSFET (GPIO 27)	IR front-end power gate	Held active so the always-on receiver keeps listening.
Cellular Modem (Currently not working and optimized)	Quectel M95R	Connectivity fallback	UART (AT commands), shares the radar UART pins — see Note.
Status Indication	RGB LED (33/32/4) + status LED (2)	System / presence state	Common-anode RGB; per-state colour and blink patterns.
User Input	Momentary push button (GPIO 13)	AP entry / exit	Short press = enter AP config; long hold = exit AP.

Pin Map (from include/Config.h)

Function	GPIO	Function	GPIO
Config button	13	Status LED	2
IR MOSFET power	27	IR transmit	18
IR receive	19	RGB Red / Green / Blue	33 / 32 / 4
Radar UART RX / TX	16 / 17	GSM UART RX / TX	16 / 17

Note — UART sharing: The radar and the GSM modem are mapped to the **same UART pins (16/17)**. Currently, GSM Module is not used so that's why it has not been tested and optimized accordingly.

Power System

- **Supply:** Mains-powered (USB / DC adapter). The device is continuously powered and does **not** run on a battery, so there is no sleep/duty-cycle power budget; the design favors always-on responsiveness, a hardware/task watchdog, and fast network reconnection.
- **IR front-end:** Powered through a MOSFET (GPIO 27) which is held active so the always-on IR receiver can continuously detect manual remote presses.

Functional Scope (Firmware)

The firmware scope is **automation-first** and **resilience-first**. The following behaviors constitute the delivery baseline.

Automation inputs

- **Radar presence** — occupancy drives the AC on/eco/off state machine.
- **7-day schedule** — per-day, up to 7 time segments, each with target temperature and optional per-segment radar / eco-temp / eco-time / off-time overrides.
- **Cloud commands (MQTT)** — temperature/power/radar/eco/schedule/OTA/dynamic-IR commands.
- **Manual remote** — a foreign IR remote press is detected, reported, and reconciled.

Automation outputs

- IR AC commands (learned per-temperature buttons, with a universal AC-protocol fallback).
- MQTT telemetry: command ACKs, automation events, health alerts, periodic status.
- Status/presence LED indication.

Operating modes / control sources

Currently, the device is running on "Schedule is king" Principle except one exemption (user input Respect). There is a dashboard developed for this purpose. You can see via visiting:

<http://107.172.137.233/dashboard/> (required credentials)

1. Offline mode

When device starts and has some credentials of Wi-Fi, it would wait for 1 minute for Wi-Fi and MQTT Connectivity, if it does not connect with Wi-Fi or its clock does not get any time, device would go into Offline mode in which radar is on by default and ac will be controlled using radar presence.

2. Online mode

For online mode understanding there are some basic functionalities you need to understand:

Enforcement Mode — ON vs OFF

Controlled from the dashboard/cloud (enforcement_control); the setting is saved in flash.

- **ON (default):** Every 3 minutes the device re-sends its current state (ON / ECO / OFF) and temperature to the AC, so any missed or garbled IR frame is automatically corrected on a timer.
- **OFF:** The periodic 3-minute re-assertion stops. A missed frame is **no longer auto-corrected on a timer** — the state is only re-applied when a schedule change, a radar event, or a detected manual remote press occurs.

Important: The enforcement toggle disables **only** the periodic timer. The immediate "manual remote press → revert" reaction is a separate path and stays active regardless of this setting (as long as Continuous IR Receiving is ON).

Continuous IR Receiving — ON vs OFF

Controlled from the dashboard/cloud (remote_ir_control); the setting is saved in flash. (The receiver hardware is always draining its buffer; this toggle only decides whether a detected press is *acted on*.)

- **ON (default):** Foreign remote presses are detected and logged to the dashboard "Remote Activity" card (with a counter). If the press is on **your AC's** protocol **and** the user changed **power or temperature** away from the schedule, the device immediately reverts the AC back to the intended (schedule/automation) state. Fan / swing / mode-only presses — where power and temperature are unchanged — are **respected** (no command sent, the user's adjustment sticks).
- **OFF:** Detected presses are ignored — nothing is logged, no counter increment, and no revert.

Combined behavior (the full matrix)

Continuous IR	Enforcement	What the device does on a manual remote change
ON	ON	Logs it, reverts immediately to the scheduled state, and the 3-min timer re-asserts in the background. (Strictest "Schedule is King".)
ON	OFF	Logs it and still reverts immediately (manual-revert is independent of the timer), but there is no background 3-min correction.
OFF	ON	Ignores the press at the moment it happens, but the 3-min timer overwrites the user's change at the next tick (correction is delayed, not immediate).
OFF	OFF	Does not revert at all — the schedule state is treated as read-only. Instead the manual power/temp change is reported to the cloud (manual_remote_change, de-duplicated) and the AC is left exactly as the user set it. This is the full "respect user input" mode — the one exemption to "Schedule is King".

With those two functionalities in place, Online Mode works as follows:

Schedule-driven control (primary)

Once the clock is synced, the schedule makes the first authoritative decision and remains in charge ("Schedule is King"). For the current day, the device evaluates the active time segment every minute and drives the AC to that segment's target temperature. Outside all configured segments, the AC is turned **OFF**. A segment can also carry its own radar, eco-temperature, eco-time, and off-time overrides.

Radar energy-saving (within the schedule)

When radar automation is enabled inside a segment, the radar refines the schedule's baseline to save energy:

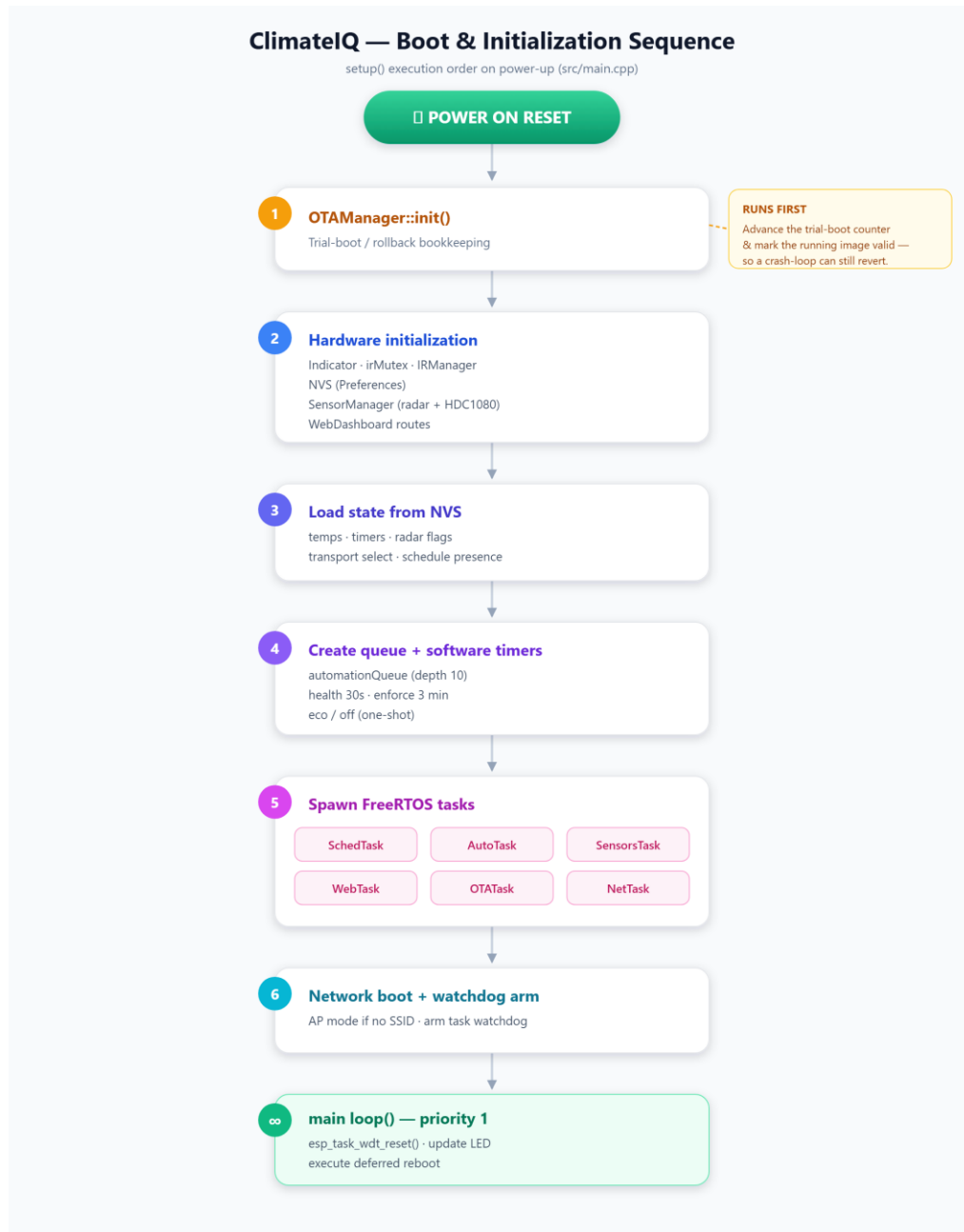
- **Room occupied** → AC **ON** at the normal (scheduled) temperature.
- **Room empty for the eco delay (default 2 min)** → AC steps to the **ECO** temperature.
- **Room empty for the off delay (default 5 min)** → AC is turned **OFF**.
- **Someone re-enters** → AC returns to **ON** at the normal temperature.

To prevent the just-stopped AC's own airflow from being misread as motion, the radar is briefly ignored after every OFF (the "flap-delay" window, default 10 s).

User-input respect (the one exemption). This is where Continuous IR Receiving matters. If the user presses the physical AC remote, the device detects it, reports the manual override to the cloud, and respects that the user has acted — re-asserting the intended state immediately instead of waiting for the next enforcement tick. Likewise, a manual command from the dashboard is honored; a one-shot temperature change made while inside a schedule is sent to the AC without overwriting the stored scheduled setpoint, so radar still re-enters at the correct temperature once the room empties and refills.

2. High-Level System Architecture

ECO PULSE boots into a deliberate initialisation order (OTA rollback bookkeeping **first**, so a crash-looping image can always be reverted), loads persisted settings from NVS, creates the shared event queue / timers / IR mutex, then spawns the FreeRTOS tasks that perform all real work. The main loop() runs at low priority and only feeds the watchdog, refreshes the LED, and executes any Deferred reboot.



At steady state the system is **event-driven**: hardware/sensor changes and timers post events to the automation queue or notify the sensor task, and each task reacts within its responsibility boundary. The supervisory loop never performs functional work.

3. Firmware Layer Architecture

The firmware is organised into clear responsibility layers. Application-level managers never touch hardware registers directly; they call abstraction/service modules, which in turn use vendor drivers and the platform layer.

1. Application Layer (product logic)

Module	Responsible for
AutomationManager	The AC state machine (OFF / ON-NORMAL / ON-ECO); the single executeACCommand() choke point; eco/off/enforce event handling; presence-time accounting.
ScheduleManager	7-day × 7-segment scheduler in NVS; minute-aligned evaluation; boot catch-up; offline failsafe; schedule-update ingest.
CommandProcessor	Inbound MQTT JSON command vocabulary → AC / schedule / radar / eco / OTA / dynamic-IR actions.
OTAManager	Server-pull firmware update with trial-boot rollback and phase reporting.
HealthManager	Periodic health checks (sensor staleness, low heap), reset-reason reporting.

2. Service / Device-Abstraction Layer

Module	Handles
SensorManager	Sole owner of the radar UART + IR receiver; radar/HDC polling; radar maintenance (calibrate/reset/range/no-one window); IR learn; presence eventing.
IRManager	AC IR transmit (learned button + universal fallback); always-on remote listener; learning; dynamic codes; all transmits serialised by <code>irMutex</code> .
NetworkManager	Thin transport abstraction; dispatches publish/telemetry to the active transport based on <code>switch_gsm_wifi</code> .
WebDashboard	SoftAP portal + local dashboard (config, IR learn, radar maintenance, dev diagnostics, local OTA).
Indicator	RGB/status/presence LED patterns; debounced config button.

3. Transport Layer

Module	Handles
WiFiManager	Wi-Fi STA + asynchronous MQTT (AsyncMqttClient); NTP sync; LWT; back-off reconnect; telemetry.
GSMManager	GSM/cellular MQTT via raw AT commands (QMTOPEN/QMTCONN/QMTPUB); RTC sync from the cell network.

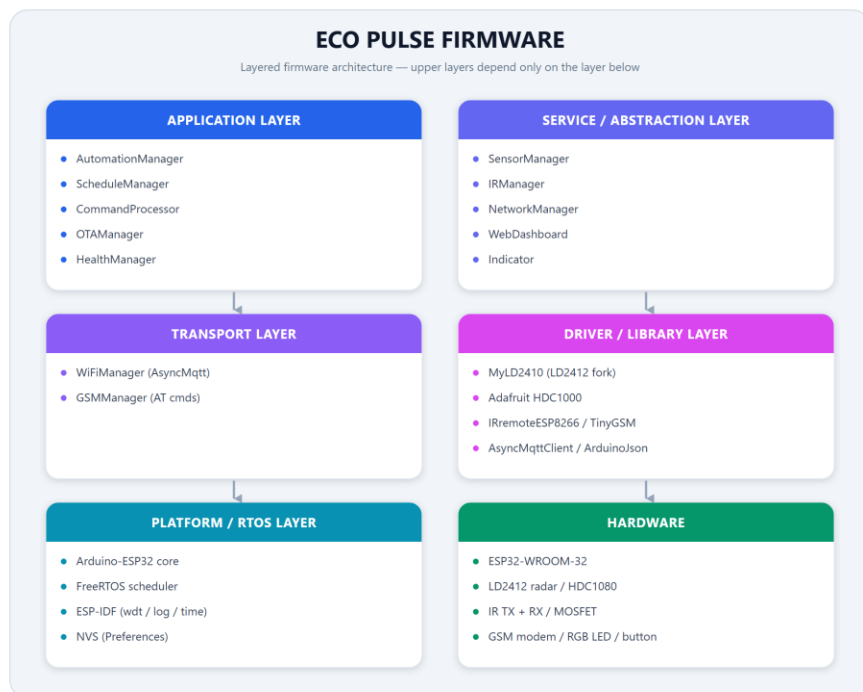
4. Driver / Library Layer

MyLD2410 (LD2412 fork) · Adafruit HDC1000 · IRremoteESP8266 · TinyGSM · AsyncMqttClient · ArduinoJson.

5. Platform Layer (foundation)

Arduino-ESP32 core · FreeRTOS (tasks, queues, software timers, task notifications) · ESP-IDF services (esp_task_wdt, esp_log, gettimeofday) · NVS via Preferences (namespace ir_data).

Visual Representation of the Layers



4. RTOS Task and Thread Architecture

ECO PULSE runs on the Arduino-ESP32 FreeRTOS port. The design is **event-driven**: hardware and sensor changes generate events that are processed asynchronously by dedicated tasks. Tasks are isolated by responsibility and communicate through a message queue, FreeRTOS software timers, and direct task notifications. The single shared state object (sysData) uses `std::atomic` fields for any value touched by more than one task; all other fields follow a documented single-writer convention.

Task-by-Task Definition

1. Schedule Task — `SchedTask`

Responsibility: Minute-aligned schedule evaluation; boot catch-up of the AC state; offline failsafe (force radar control after 60 s offline with no clock).

Priority / Core / Stack: 4 / Core 1 / 4096 B.

Inputs: NVS schedule segments, system clock (NTP/GSM).

Outputs: `executeACCommand()`, radar-mode toggles, ACKs.

2. Automation Task — `AutoTask`

Responsibility: Drains `automationQueue`; drives the AC state machine; emits `auto_event` telemetry; applies the "schedule is king" and "radar block outside schedule" policies.

Priority / Core / Stack: 4 / Core 1 / 4096 B.

Queue: `automationQueue` (10 × `SystemEvent`).

Event types: presence change, eco/off/enforce triggers, manual override.

3. Sensors Task — `SensorsTask`

Responsibility: Sole owner of the radar UART and IR receiver. Polls radar + HDC1080; runs the always-on remote listener; performs radar maintenance on notification; reads boot range; accounts presence time.

Priority / Core / Stack: 3 / Core 1 / 4096 B (handle captured for notifications).

Notification bits: (1<<1) calibrate · (1<<2) factory reset · (1<<3) set range · (1<<4) IR learn · (1<<5) set no-one window.

4. Web Task — `WebTask`

Responsibility: Services the web server / SoftAP portal and deferred AP exit.

Priority / Core / Stack: 1 / Core 0 / 4096 B.

5. OTA Task — `OTATask`

Responsibility: Idle until an `ota_update` command queues a job, then downloads + flashes the inactive partition and reboots.

Priority / Core / Stack: 2 / Core 1 / 10240 B (covers the TLS/HTTPUpdate chain).

6. Network Task — `NetworkTask`

Responsibility: Services the active transport (Wi-Fi or GSM): reconnection back-off, MQTT servicing, periodic telemetry.

Priority / Core / Stack: 1 / Core 1 / 8192 B.

Supervisory loop (`loop()`) — Priority 1, Core 1: watchdog reset, LED refresh, deferred reboot.

Software Timers

Timer	Period	Type	Action
healthTimer	30 s	periodic	Sensor staleness + low-heap checks (HealthManager).
enforceTimer	3 min	periodic	Queue EVENT_ENFORCE_TRIGGER (re-assert AC state).
ecoTimer	configurable	one-shot	Queue EVENT_ECO_TRIGGER (room empty → eco).
offTimer	configurable	one-shot	Queue EVENT_OFF_TRIGGER (room empty → off).

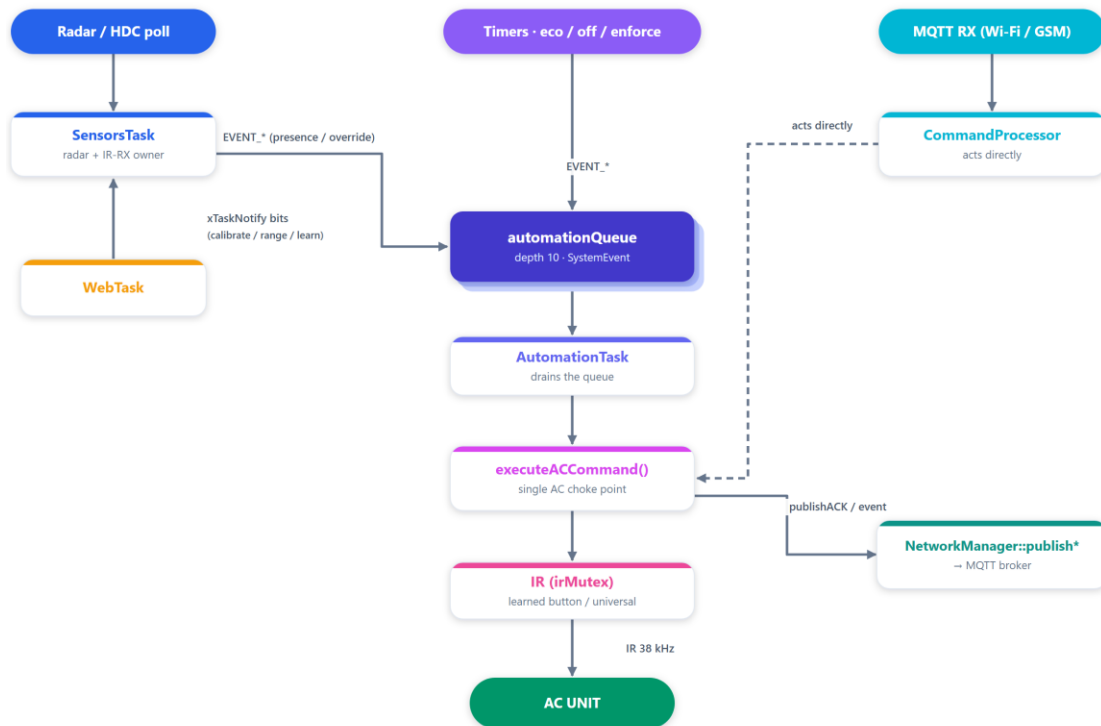
Priority Model

Priority	Tasks
4 (highest)	Schedule, Automation
3	Sensors
2	OTA
1 (lowest)	Web, Network, main loop()

Inter-Task Communication

Eco Pulse — Inter-Task & Event Data Flow

Solid = event posted via automationQueue · Dashed = direct function call



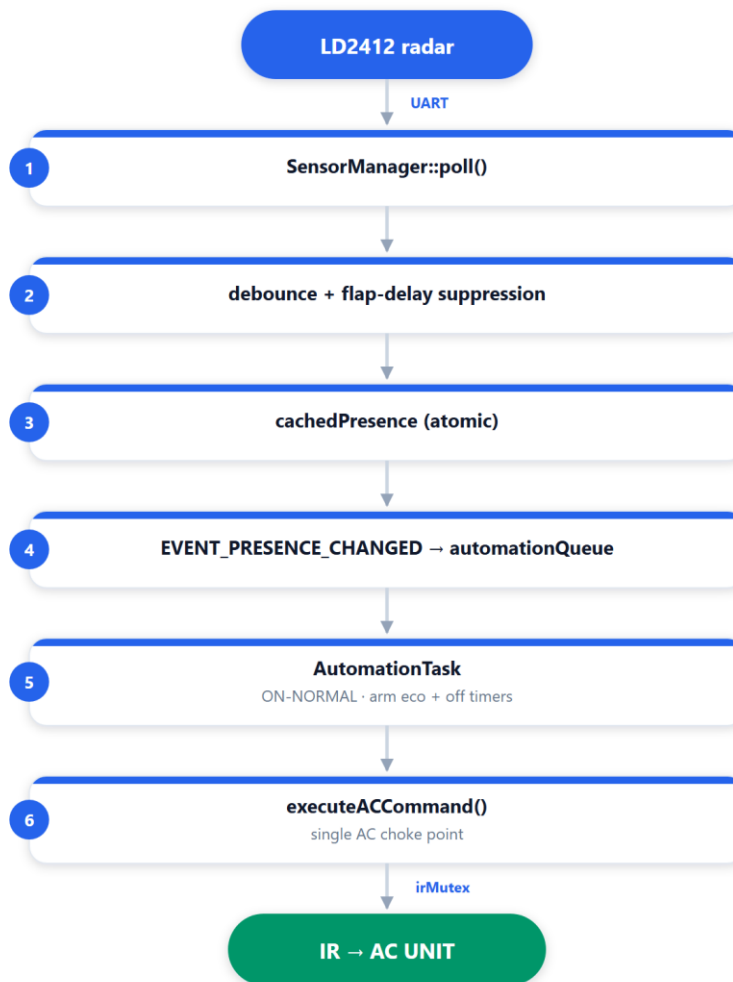
- **Queue:** automationQueue — the single event channel into the automation state machine.
- **Mutex:** irMutex — serialises every IR transmit across tasks.
- **Notifications:** xTaskNotify() bitmask wakes the sensor task for radar/IR maintenance (the web/network tasks never touch the radar UART directly).
- **Shared atomics:** presence, radar mode, range/no-one mirrors, AC state, schedule flags, and presence accumulator live in sysData.

5. Data Flow Architecture

This section describes how data moves through ECO PULSE. In general: sensors and the cloud generate events, the automation/schedule logic decides the AC action, IR drives the unit, and telemetry is published back to the broker.

Sensor Data Path (presence → AC)

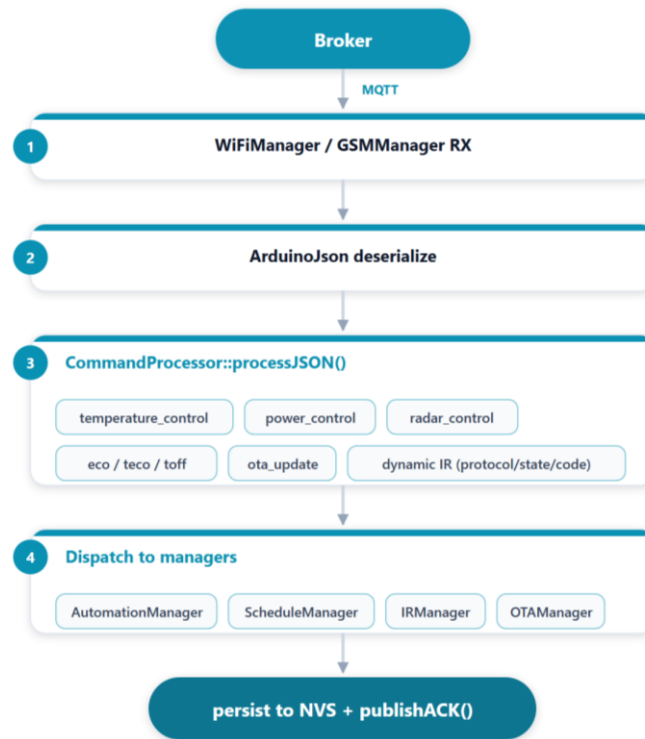
1 · Sensor Presence → AC Control



- **HDC1080:** sampled ~every 10 s on the sensor task; temperature/humidity stored for telemetry; NaN reads trigger I²C re-init after a threshold.
- **Radar per-gate energy:** moving/static energy for all valid gates is mirrored into sysData for the dashboard developer feed (diagnostic only).

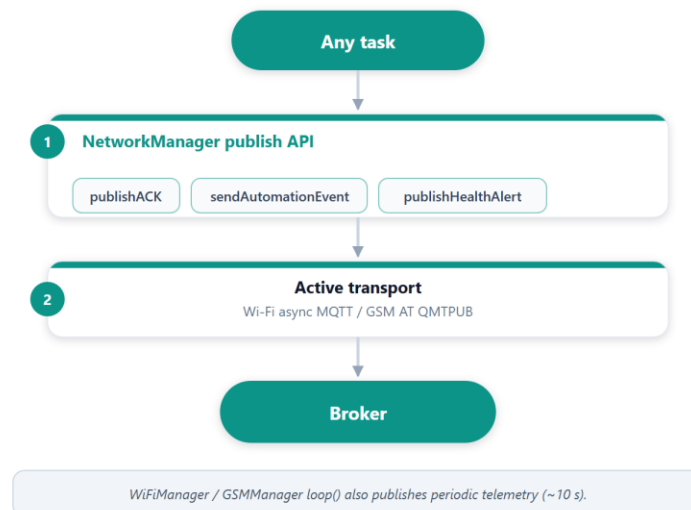
Command Data Path (cloud → device)

2 · Cloud Command → Device Action



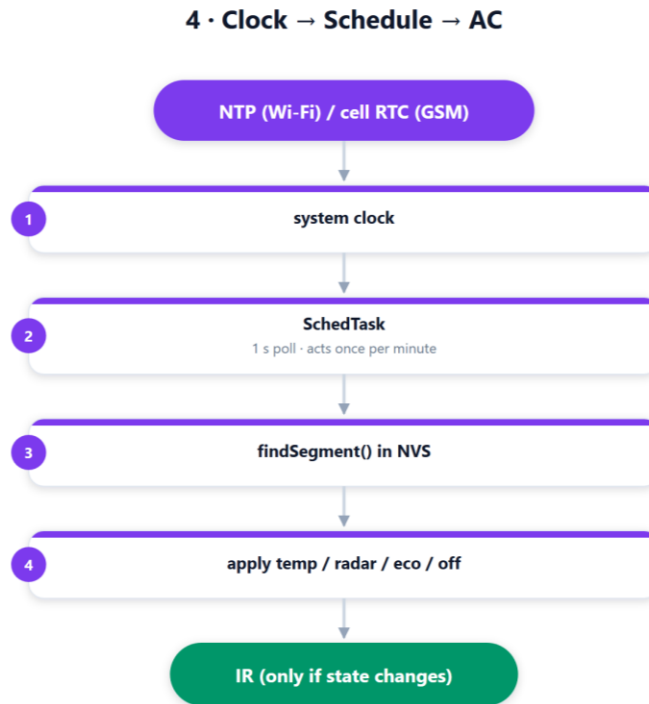
Status / Telemetry Data Path (device → cloud)

3 · Telemetry / Events → Cloud

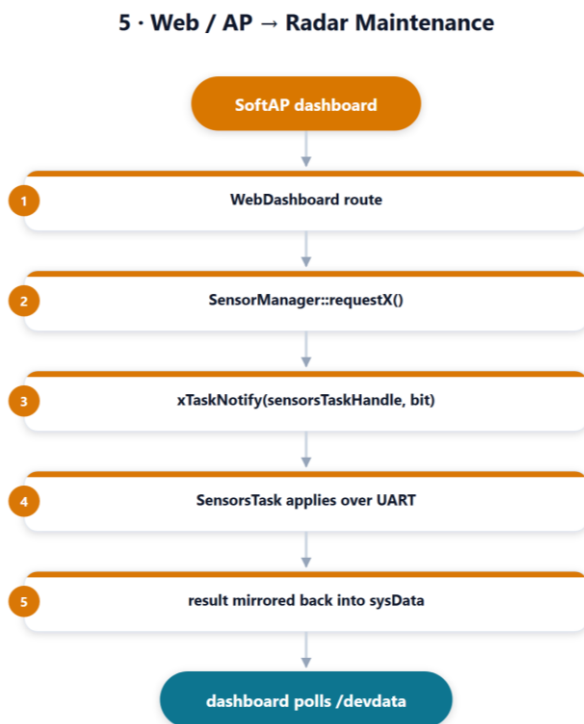


auto_event codes: **1000** normal-on · **2000** eco · **3000** off · **4000** manual override.

Schedule Data Path (clock → AC)



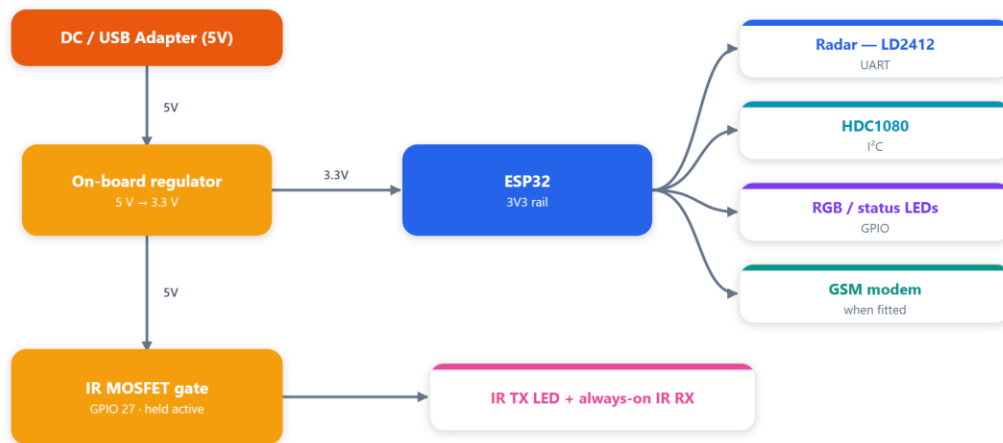
Local Path (web/AP → radar/IR, no cloud)



6. Power Architecture

ECO PULSE is a **mains-powered, always-on** controller; unlike battery wearable there is no deep-sleep/duty-cycle power budget. The power architecture therefore focuses on **clean power domains, IR front-end gating, and uptime resilience** rather than energy harvesting.

Eco Pulse — Power Architecture



Design points

- **No sleep states:** the device stays awake to keep MQTT, the radar stream, and the always-on IR receiver live; resilience is provided by the watchdog and reconnection logic, not power gating.
- **IR front-end power:** controlled by the MOSFET on GPIO 27, held active so the receiver keeps listening for manual remote presses (which drive the manual-override path).
- **Reconnection over back-off-to-sleep:** because the device is mains-powered, the Wi-Fi/MQTT back-off is **capped at 60 s** (MAX_BACKOFF_MS) — it keeps retrying frequently rather than backing off for minutes.

Assumption: Power-rail regulation and protection are handled on the carrier PCB. This firmware assumes a stable 3V3 supply and does not implement battery-management or brown-out mitigation beyond the ESP32's built-in brown-out detector and the task watchdog.

7. Communication Architecture

External Communication

Transport selection. NetworkManager dispatches all publish/telemetry calls to the active transport based on `switch_gsm_wifi` (true = Wi-Fi, false = GSM). Both transports mirror the same publish API (`publishACK`, `sendAutomationEvent`, `publishHealthAlert`).

Interface	Stack	Role
Wi-Fi + MQTT	AsyncMqttClient over Wi-Fi STA	Primary transport; LWT, subscribe, telemetry, async RX → CommandProcessor.
GSM + MQTT	Raw AT commands (QMTOPEN/QMTCONN/QMTPUB)	Fallback transport; PDP bring-up, RTC sync from the cell network.
NTP	pool.ntp.org + Google + Cloudflare	Wall-clock sync for the schedule (multi-server for filtered networks).
OTA (HTTP/S)	HTTPUpdate pull	Firmware download from the deployment server.

MQTT broker: 107.172.137.233:1883 (plaintext). Device identity is the Wi-Fi STA MAC.

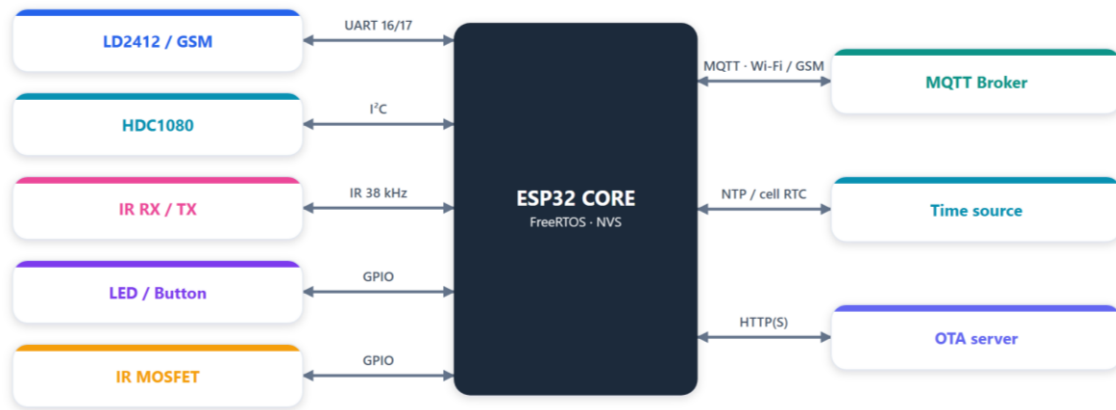
Inbound command vocabulary (CommandProcessor):

- `temperature_control` — schedule update (with segments) or setpoint / ir mapping.
- `power_control` — `power_status` on/off.
- `radar_control` — enable/disable radar automation (manual override).
- `eco / teco / toff` — eco setpoint and eco/off delays.
- `ota_update` — `url`, `version`, optional `force`.
- Dynamic IR — top-level protocol with `state[]` or `code`.

Internal Communication Interfaces

Bus	Used for
UART (Serial1, 16/17)	LD2412 radar config + data stream (or the GSM modem — mutually exclusive).
I ² C	HDC1080 temperature/humidity (addr 0x40).
IR (38 kHz)	AC command transmit (GPIO 18) + always-on receive (GPIO 19).
GPIO	RGB/status LED, config button, IR MOSFET gate.
NVS (flash)	Persisted settings: temps, timers, radar flags, schedule, IR codes, OTA bookkeeping.

Eco Pulse — Communication Architecture

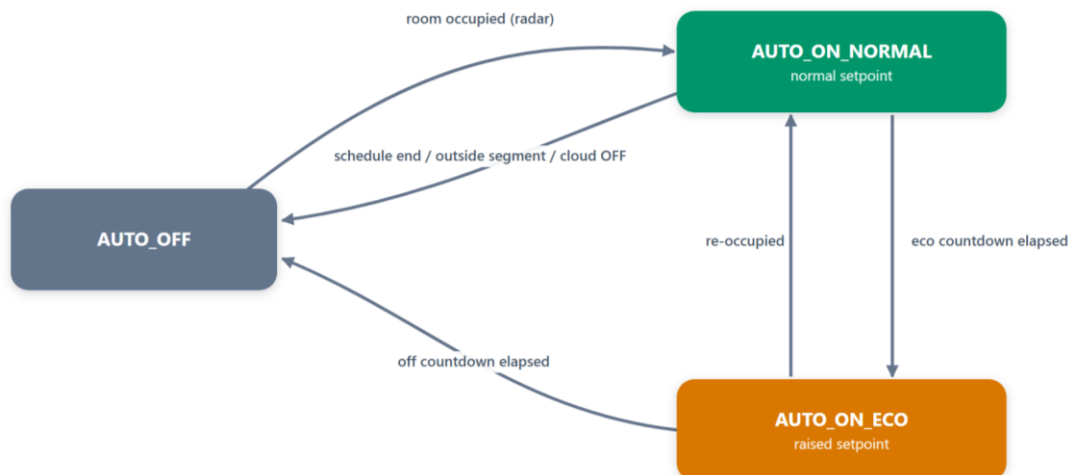


8. Mode State Machine Design

ECO PULSE runs two cooperating state models: the **AC automation state machine** (what the AC is doing) and the **system/connectivity state** (what the device is doing on the network, used for LED indication). On top of these sit the control-source policies that decide *who* may drive the AC.

AC Automation State Machine (AutoState)

Eco Pulse — AC Automation State Machine



- **AUTO_OFF** — AC off.
- **AUTO_ON_NORMAL** — AC on at the scheduled/normal setpoint (occupant present).

- **AUTO_ON_ECO** — AC at the raised eco setpoint (room empty for TEcoTime); after TOffTime the room-empty timer drives it to AUTO_OFF.

Every transition is applied through `executeACCommand()`, which sends IR (learned button → universal fallback), resets the enforce timer, opens the radar **flap-delay** blind spot on an OFF, flashes the indicator, and publishes an ACK.

Control-Source Policy (who may drive the AC)

- **Schedule is king:** at boot, radar may not drive the AC until the schedule has made its first authoritative decision (`scheduleBootDone`). The offline failsafe is the sole exception.
- **Radar block outside schedule:** when a schedule exists and the clock is valid but the current time is outside all segments, radar events are dropped (unless the "radar outside" policy opts in).
- **Manual override:** a cloud `radar_control` command or a detected physical remote press overrides and is reconciled (the manual-override path re-asserts the intended state immediately).
- **Enforce re-assertion:** every 3 minutes the intended AC state is re-sent to correct a missed/garbled IR frame (suppressible per-feature via the backend toggle).

System / Connectivity State (SystemState, drives the LED)

`SystemState` is an 8-value enum (`SharedState.h`) held in `sysData.currentState`. Its sole job is to drive the RGB status LED in `Indicator::update()`; each value maps to a distinct colour/blink so the device's connection status is readable at a glance:

State	Meaning	Status LED
<code>SYS_BOOTING</code>	Power-on, before any connection attempt	White, solid
<code>SYS_AP_MODE</code>	SoftAP configuration portal active	Cyan, blink (500 ms)
<code>SYS_WIFI_CONN</code>	Wi-Fi connecting / reconnecting	Blue, blink (500 ms)
<code>SYS_WIFI_OK</code>	Wi-Fi and MQTT up — actually delivering data	Green, slow (5 s)
<code>SYS_MQTT_DOWN</code>	Wi-Fi associated but broker session down (not sending)	Amber, blink (500 ms)
<code>SYS_GSM_CONN</code>	GSM modem connecting	Magenta, blink (500 ms)
<code>SYS_GSM_OK</code>	GSM + MQTT up	Magenta, slow (5 s)
<code>SYS_ERROR</code>	Unrecoverable / error indication	Red, fast (200 ms)

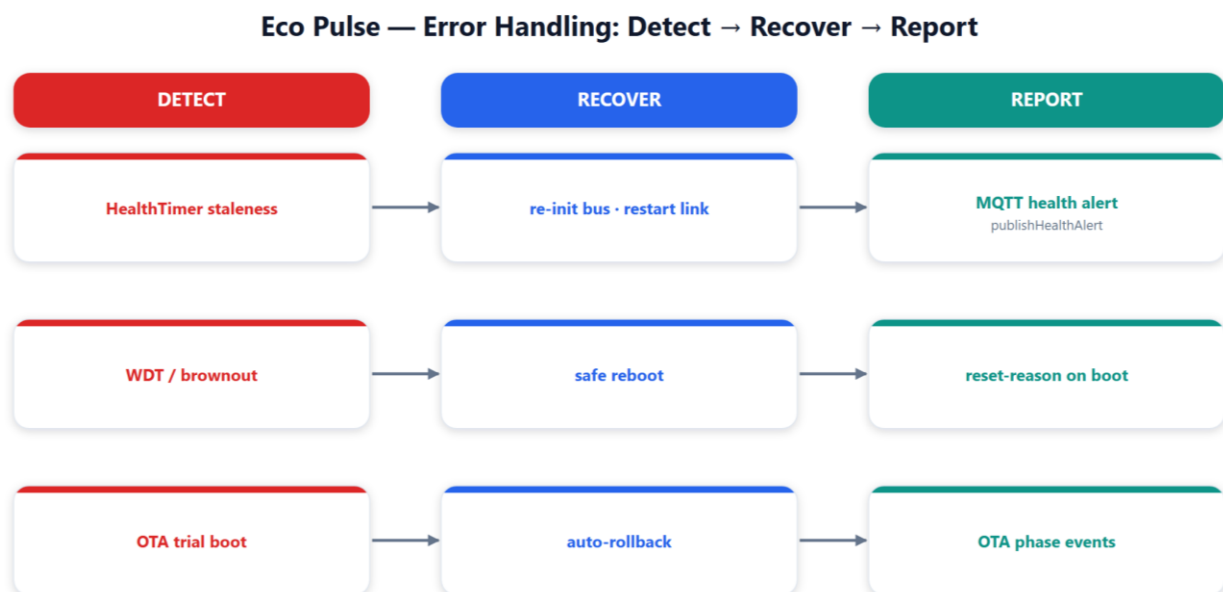
- Offline Failsafe (LED override — not a `SystemState`)
- The offline failsafe is **not** a member of the `SystemState` enum. It is a separate boolean field,
- `sysData.isOfflineFailsafeActive`, set by the schedule task. In `Indicator::update()` it is checked
- **before** the `SystemState` switch and, when set, forces **solid magenta** and returns early — so it

- **overrides whatever `currentState` actually is** rather than being a state the machine transitions into.
- It engages when the broker is unreachable **and** NTP never synced for 60 s: the schedule task forces
- radar control so the room is still automated locally (solid-magenta LED). When the clock later syncs,
- the schedule re-asserts authority ("schedule is king") and the flag clears, returning the LED to the
- normal SystemState colour.
- **Note:** Because SYS_GSM_CONN / SYS_GSM_OK also use magenta (blink / slow), the offline failsafe is deliberately **solid** magenta to remain distinguishable from the GSM states.

9. Error Handling Architecture

1. Overview

ECO PULSE implements a layered fault-detection and recovery strategy designed for an unattended, always-on appliance: **Detect → Recover → Report**, with a hardware/task watchdog as the final backstop and OTA trial-boot rollback protecting against bad firmware.



2. Error Categories

Critical (reboot / rollback)

MCU crash / hard fault, task watchdog timeout (120 s), brown-out.

Crash-looping new firmware after OTA → automatic rollback to the previous partition.

Major (subsystem recovery)

Radar UART stall / "frozen radar" (no frame for RADAR_STALE_MS = 30 s) → radar link re-init.

HDC1080 repeated NaN reads ($\geq$ HDC_REINIT_THRESHOLD) $\rightarrow$ I²C re-init.

MQTT/Wi-Fi/GSM disconnection $\rightarrow$ back-off reconnect (cap 60 s).

Minor (filter / ignore)

IR repeat frames / noise / our own transmissions $\rightarrow$ filtered by the remote listener.

Radar false motion from the just-stopped AC airflow $\rightarrow$ suppressed by the flap-delay window.

Transient telemetry publish failures $\rightarrow$ retried on the next interval.

3. Recovery Mechanisms

Domain	Mechanism
Radar	Staleness detection $\rightarrow$ attemptRadarRecovery() (UART re-begin + enhanced mode); config-mode resync before maintenance; BT lockout to prevent external BLE hijack.
Climate sensor	attemptHDCRecovery() (Wire re-init at 0x40) after repeated NaN reads.
Network	Exponential back-off reconnection (capped); MQTT LWT for clean "offline" signalling; periodic RTC resync.
System	Task watchdog (esp_task_wdt, 120 s) $\rightarrow$ reset; deferred-reboot mechanism; reset-reason reported on next boot.
Firmware	OTA trial-boot: a new image is committed only after MQTT reconnects; otherwise it auto-reverts after OTA_MAX_TRIAL_BOOTs (3) boot-loops (optional time-based revert).
Automation	Offline failsafe hands control to radar; enforce timer re-asserts intended AC state every 3 min.

4. Fault Reporting

- **MQTT health alerts** (publishHealthAlert) — event + detail.
- **OTA phase events** — downloading / success / failed / rolled_back / skipped / aborted (MQTT is intentionally down during the flash, so progress is reported as phases).
- **Serial logs** — per-module ESP-IDF ESP_LOGx TAGs at INFO for app code, ERROR for libraries.
- **Reset reason** — decoded and reported at boot (HealthManager::getResetReason).

10. Security Architecture

The current production baseline favours field-serviceability; the following describes the implemented model and its known gaps (carried into §11).

Local Access (SoftAP / Web Dashboard)

- **SoftAP** protected by a WPA2 passphrase (AP_PASSWORD).
- **Dashboard** behind HTTP basic auth (WWW_USERNAME / WWW_PASSWORD); a separate developer/diagnostic password (DEV_PASSWORD) gates the dev feed.

Note: Default credentials are shipped in Config.h. These must be changed/rotated per deployment; they are a known hardening item.

Cloud Transport (MQTT)

- MQTT runs on **plaintext port 1883** with a per-device client id (Wi-Fi MAC) and a Last-Will message. There is no TLS or broker-side mutual authentication in this baseline.

Firmware Updates (OTA)

- **Integrity:** the ESP32 ROM bootloader verifies the image SHA-256, and the trial-boot mechanism guarantees a bad image is reverted.
- **Transport:** the deployment server is plain HTTP, so `setInsecure()` is used. A root-CA pin (OTA_ROOT_CA) is supported for HTTPS but is empty in this baseline — so the channel is **not** protected against an active MITM serving a validly-signed malicious binary.

Radar Hijack Protection

- The radar's Bluetooth is **disabled** and the setting persisted in the radar's own flash, so no external BLE app can seize the radar's config mode and freeze the UART stream. A compile-time `RADAR_ENABLE_BT` toggle re-enables BT only for bench testing.

11. Design Assumptions & Risk Analysis

1. Design Assumptions

Hardware

ESP32-WROOM-32 (DoIT DevKit) is the final MCU.
HLK-LD2412 (14-gate) is the presence sensor, driven via the local MyLD2410 fork.
HDC1080 is the climate sensor (I²C 0x40).
The device is mains-powered and continuously online.
Radar and GSM share UART pins 16/17 (mutually exclusive transports).

Firmware

Arduino-ESP32 + FreeRTOS is the base platform; NVS (Preferences, namespace `ir_data`) persists all settings.
The schedule depends on a valid wall clock (NTP or cell RTC); the offline failsafe covers the gap.
IR control assumes the target AC is learnable, with a universal protocol as fallback.

Behaviour

The schedule is the authoritative baseline; radar provides energy-saving within it.
The cloud back-end and command front-end exist and speak the documented MQTT vocabulary.

2. Risk Analysis

2.1 MQTT plaintext + default credentials (HIGH)

Risk: Port 1883 is unencrypted; default web/AP credentials ship in firmware.
Impact: Eavesdropping / command injection / unauthorised local config.
Mitigation: Move to TLS MQTT and per-device credentials; force credential rotation on provisioning; restrict the dev feed.

2.2 OTA over plain HTTP (HIGH)

Risk: `setInsecure()` with an empty root CA; integrity relies on ROM SHA-256 only.
Impact: An active MITM could serve a validly-signed malicious binary.
Mitigation: Serve OTA over HTTPS with a pinned `OTA_ROOT_CA`; sign release manifests.

2.3 Radar / GSM UART pin conflict (MEDIUM-HIGH)

Risk: Radar and GSM cannot run together (shared pins 16/17).
Impact: A GSM-fallback unit loses presence automation; a Wi-Fi unit cannot use GSM.
Mitigation: Treat as a deployment-time SKU choice; revisit pin allocation in the next PCB rev.

2.4 Radar stall / "frozen radar" (MEDIUM)

Risk: External BLE config or a desynced config state machine freezes the UART stream.

Impact: Presence automation stops.

Mitigation: BT lockout (persisted), 30 s staleness watchdog + UART recovery, config resync before every maintenance operation. *(Implemented.)*

2.5 Clock dependency for scheduling (MEDIUM)

Risk: NTP blocked/slow on filtered networks; no clock → no schedule.

Impact: Schedule cannot evaluate.

Mitigation: Multi-server NTP, cell RTC on GSM, and the 60 s offline failsafe handing control to radar. *(Implemented.)*

2.6 IR open-loop control (MEDIUM)

Risk: IR is fire-and-forget; the AC may miss a frame or be changed by a physical remote.

Impact: Device belief and AC reality diverge.

Mitigation: 3-minute enforce re-assertion, manual-remote detection + reconciliation, double-blast custom buttons. *(Implemented.)*

2.7 Single shared NVS namespace (LOW)

Risk: All settings share one Preferences namespace and the global handle.

Impact: Key collisions / wear if extended carelessly.

Mitigation: Maintain a documented key registry; consider namespacing by subsystem.

12. Architecture Review & Sign-Off

This architecture document represents the agreed baseline for ECO PULSE firmware **.0.15**. The modules, task model, data flows, and policies described here are implemented in the current codebase.

Tuning of parameters (eco/off timers, radar range and presence-hold window, schedule segments, reconnect back-off) is permitted without a document revision; structural or feature-level changes require an update to this document and re-approval.

Open Items Carried Forward

#	Item	Owner	Target Phase
1	TLS MQTT + per-device credentials	Firmware / Cloud	Next security revision
2	HTTPS OTA with pinned root CA	Firmware / DevOps	Next security revision
3	Resolve radar/GSM UART pin sharing	Hardware	Next PCB revision
4	Rotate default web/AP credentials per unit	Provisioning	Manufacturing
5	NVS key registry / namespacing	Firmware	Maintenance

Sign-Off

Role	Name	Signature	Date
Firmware Lead			
Hardware Lead			
Project Manager			
Client Representative			